Final Project Report

Tayarny Airlines — Flight Reservation System

Team Members

Abdelrahman Amr (9010)
Omar Waleed (8873)
Youssef Mohamed (8821)
Mohamed Hatem (8995)
Mahmoud Waleed (8958)
Abdelrahman Hussein (8976)

Course Name: Software Engineering (SSP)
Instructor Name: Dr. Yousry Taha
Submission Date: May 13, 2026

Contents

1 Project Overview **4**

2 Problem Statement **4**

3 System Scope **4**

3.1 In Scope 4

3.2 Out of Scope 5

4 Software Development Lifecycle (SDLC) **5**

4.1 Requirements Phase 5

4.1.1 Functional Requirements 5

4.1.2 Non-Functional Requirements 7

4.2 Design Phase 7

4.2.1 System Architecture 7

4.2.2 Design Decisions 8

4.2.3 Use Case Diagram 9

4.2.4 Sequence Diagram 10

4.2.5 Activity Diagrams 11

4.2.6 Class Diagram 14

4.2.7 State Machine Diagrams 15

4.3 Implementation Phase 15

4.3.1 Technologies and Tools Used 15

4.3.2 Development Overview 16

4.3.3 Key Components 16

4.4 Testing Phase 17

4.4.1 Testing Strategy 17

4.4.2 Test Cases 17

4.4.3 Verification of Correctness 17

4.5 Maintenance Phase 18

4.5.1 Maintainability 18

4.5.2 Future Improvements 18

4.5.3 Scalability and Updates 18

5 Project Management and Collaboration **19**

5.1 Trello 19
5.2 Notion 19

1 Project Overview

Tayarny Airlines is a full-stack flight reservation system that supports a complete booking lifecycle for two roles: **Customer** and **Administrator**.

Customers can search for available flights by origin, destination, and departure date, choose a cabin class (Economy/Business/First), view the seat map, select seats for travelers, and complete a booking to generate a reservation and tickets.

Administrators can manage operational data and schedules through an admin interface, including managing airports, airplanes, and flight schedules, as well as monitoring and managing users, reservations, and tickets.

The system is implemented with a Next.js frontend, a Spring Boot REST API backend secured by JWT role-based access control, and a PostgreSQL database (Dockerized for consistent deployment).

2 Problem Statement

Traditional flight booking and reservation management can be slow and error-prone when handled manually or across disconnected tools.

Customers need a fast and clear way to search flights, compare options, select seats, and confirm bookings, while the airline needs a reliable way to manage flight schedules, seat availability, and reservations without inconsistencies or double-booking.

The problem addressed by this project is building a secure, end-to-end reservation platform that:

- Provides a simple customer booking workflow from search to confirmation.
- Maintains correct seat availability under concurrent usage.
- Gives administrators controlled access to manage operational data and bookings.

3 System Scope

3.1 In Scope

- User authentication and authorization with two roles: Customer and Administrator.
- Customer features: airport autocomplete/search, flight search, viewing fare options, seat map viewing, seat selection, traveler data entry, and checkout.
- Customer features: reservation history, reservation details, ticket viewing, and cancellation.
- Admin features: manage flights, airports, airplanes, users, reservations, and tickets.
- Data persistence using PostgreSQL and API-based communication between frontend and backend.

3.2 Out of Scope

- Real payment gateway integration.
- Airline check-in, boarding pass generation, baggage handling, and airport operations.
- Refund processing and financial settlement workflows.
- Loyalty programs, promotions, and dynamic pricing.
- External integrations such as Email/SMS notifications.

4 Software Development Lifecycle (SDLC)

4.1 Requirements Phase

4.1.1 Functional Requirements

FR-1: User Signup
The system shall allow new users to create accounts using the registration form and assign the default CUSTOMER role.

FR-2: User Login
The system shall authenticate users using username and password credentials and return JWT access and refresh tokens.

FR-3: Token Refresh
The system shall allow users to obtain a new access token using a valid refresh token.

FR-4: Logout
The system shall invalidate refresh tokens during logout operations.

FR-5: Role-Based Access Control
The system shall enforce ADMIN and CUSTOMER permissions on protected endpoints.

FR-6: Airport Search and Autocomplete
The system shall provide airport search using city names, airport names, or airport codes.

FR-7: Airport Lookup
The system shall allow retrieving airport details by ID or IATA code.

FR-8: Flight Search
The system shall allow customers to search flights using origin, destination, departure date, passenger counts, and cabin class.

FR-9: View Flight Fares
The system shall display available fares, prices, and remaining seats for matching flights.

FR-10: Seat Map Viewing
The system shall allow users to view seat layouts and availability for a selected flight.

FR-11: Traveler Information Entry
The system shall allow customers to enter passenger details such as names, seat assignments, and passport information.

FR-12: Checkout and Booking Confirmation
The system shall create reservations and tickets after validating booking and payment information.

FR-13: Prevent Double Booking
The system shall lock seats during checkout to prevent concurrent seat assignment conflicts.

FR-14: Reservation History
The system shall allow customers to view paginated reservation history.

FR-15: Reservation Details
The system shall display reservation details and associated tickets.

FR-16: Reservation Cancellation
The system shall allow reservation cancellation according to business rules.

FR-17: Ticket Cancellation
The system shall allow individual ticket cancellation.

FR-18: User Profile Management
The system shall allow users to view, update, and delete profile data.

FR-19: Admin Flight Management
The system shall allow administrators to create, update, list, and delete flights.

FR-20: Admin Airport Management
The system shall allow administrators to manage airport information.

FR-21: Admin Airplane Management
The system shall allow administrators to manage airplane records.

FR-22: Admin User Management
The system shall allow administrators to view and remove user accounts.

FR-23: Admin Reservation Management
The system shall allow administrators to monitor and cancel reservations.

FR-24: Admin Ticket Management
The system shall allow administrators to manage ticket records.

4.1.2 Non-Functional Requirements

NFR-1: Performance Optimization
Typical read operations shall respond within two seconds under normal load conditions.

NFR-2: Pagination Support
Large datasets such as reservations and tickets shall support paginated retrieval.

NFR-3: Password Security
Passwords shall be stored using BCrypt hashing and never in plaintext form.

NFR-4: JWT Authentication
Authentication shall use stateless JWT-based security mechanisms.

NFR-5: Authorization Enforcement
The system shall enforce ADMIN and CUSTOMER access restrictions on APIs.

NFR-6: Input Validation
Invalid requests and malformed input shall be rejected.

NFR-7: Payment Data Protection
Raw credit card data shall not be permanently stored.

NFR-8: Usability
The booking workflow shall provide clear navigation and meaningful error messages.

NFR-9: Airport Autocomplete Usability
Airport autocomplete shall reduce manual entry errors.

NFR-10: Transactional Booking
Reservation creation, ticket generation, and seat updates shall execute atomically.

NFR-11: Seat Consistency Reliability
Database locking shall prevent inconsistent seat assignments.

NFR-12: Error Handling
The system shall return proper responses for common failure scenarios.

NFR-13: Horizontal Scalability
The backend shall support scaling through stateless authentication.

NFR-14: Deployment Consistency
Docker Compose shall provide consistent environment setup and deployment.

4.2 Design Phase

4.2.1 System Architecture

The system uses a client–server architecture with three main layers:

- **Presentation Layer:** Next.js frontend application.
- **Application Layer:** Spring Boot REST API implementing business logic.
- **Data Layer:** PostgreSQL database storing all system entities.

Communication is performed via JSON over HTTP.

4.2.2 Design Decisions

- Stateless JWT authentication.
- Role-based authorization.
- Transactional booking system.
- Concurrency control using database locking.
- DTO-based API contracts.
- Pagination support.
- Docker Compose containerization.

4.2.3 Use Case Diagram

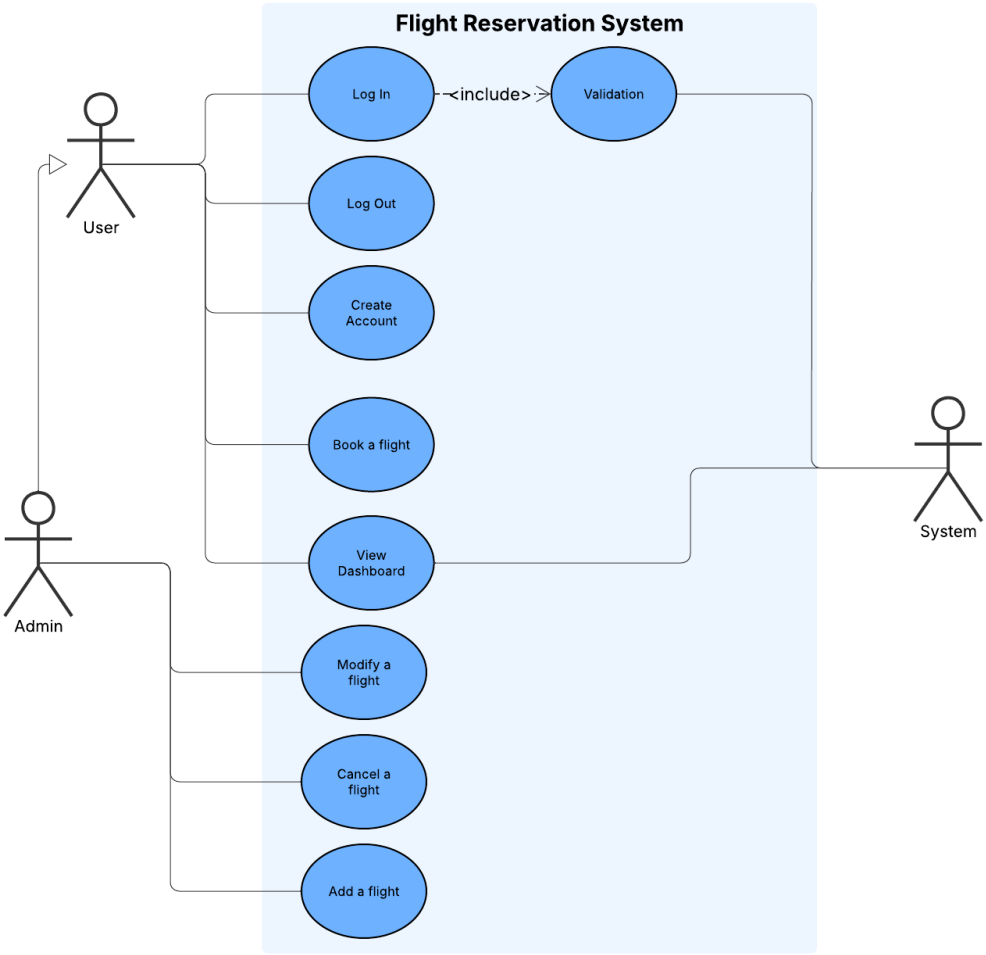

Figure 1: Use Case Diagram

4.2.4 Sequence Diagram

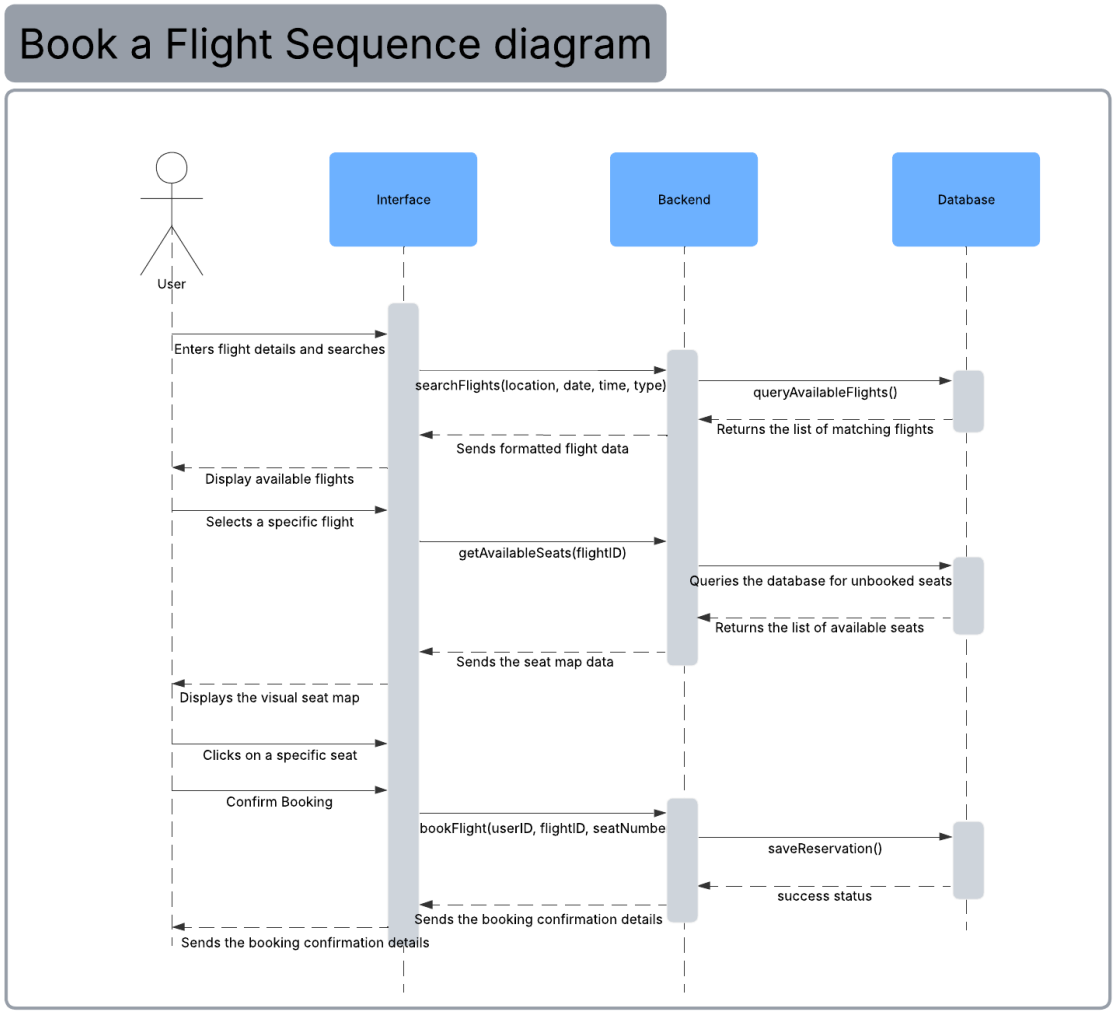

Figure 2: Sequence Diagram

4.2.5 Activity Diagrams

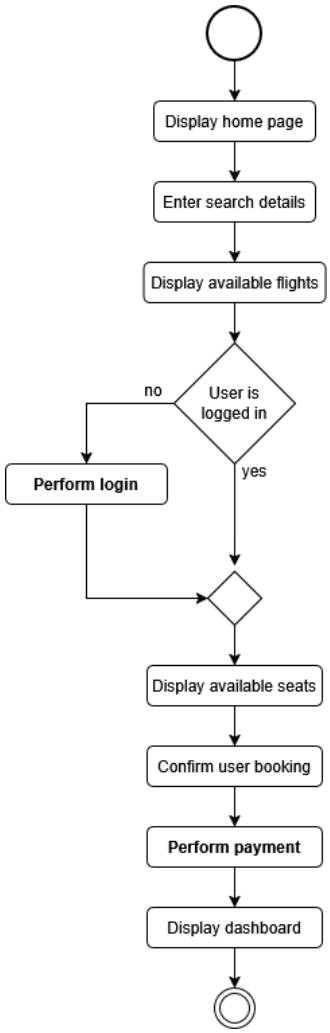

Figure 3: Activity Diagram (Book Flight)

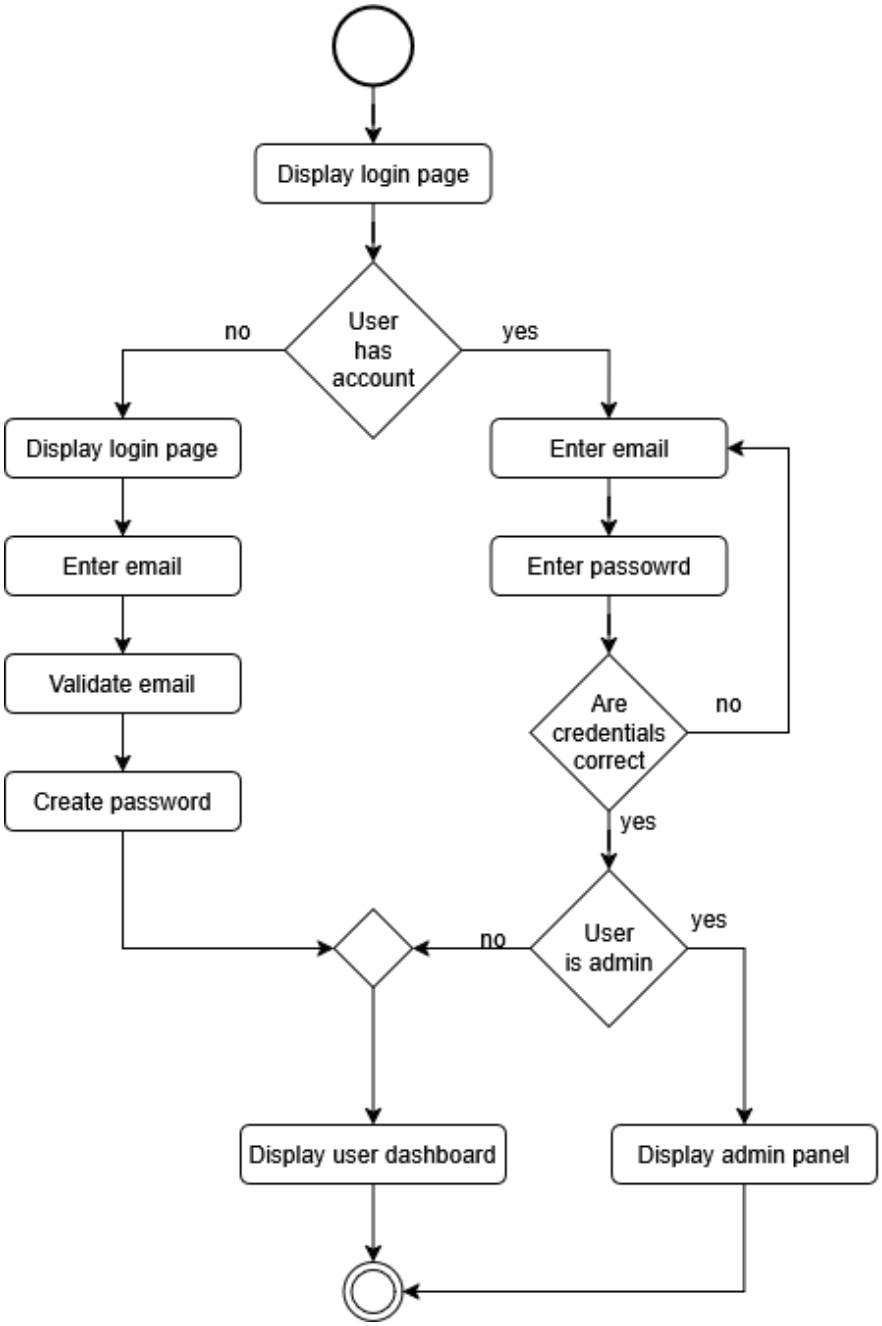

Figure 4: Activity Diagram (Login)

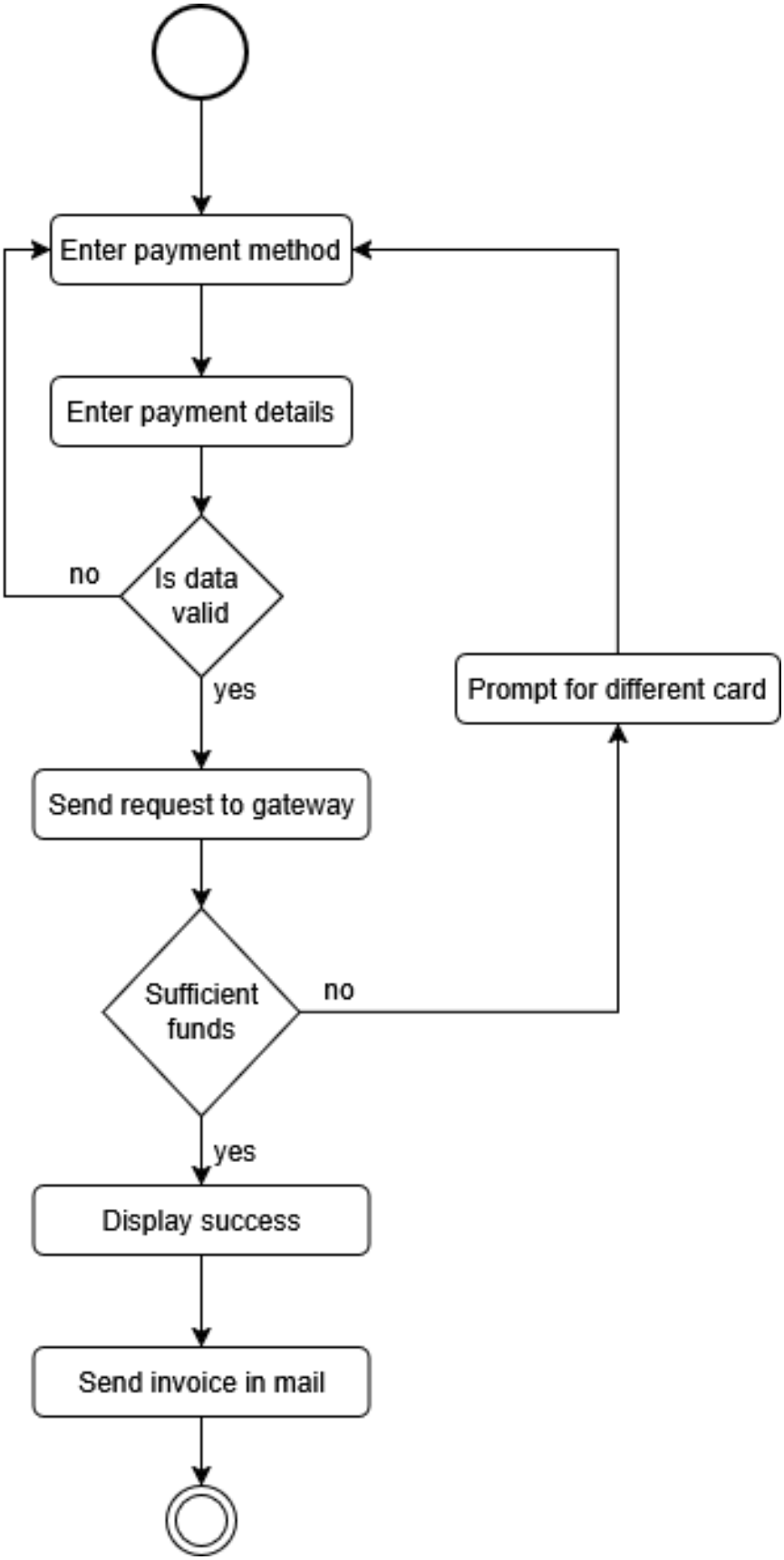

Figure 5: Activity Diagram (Payment)

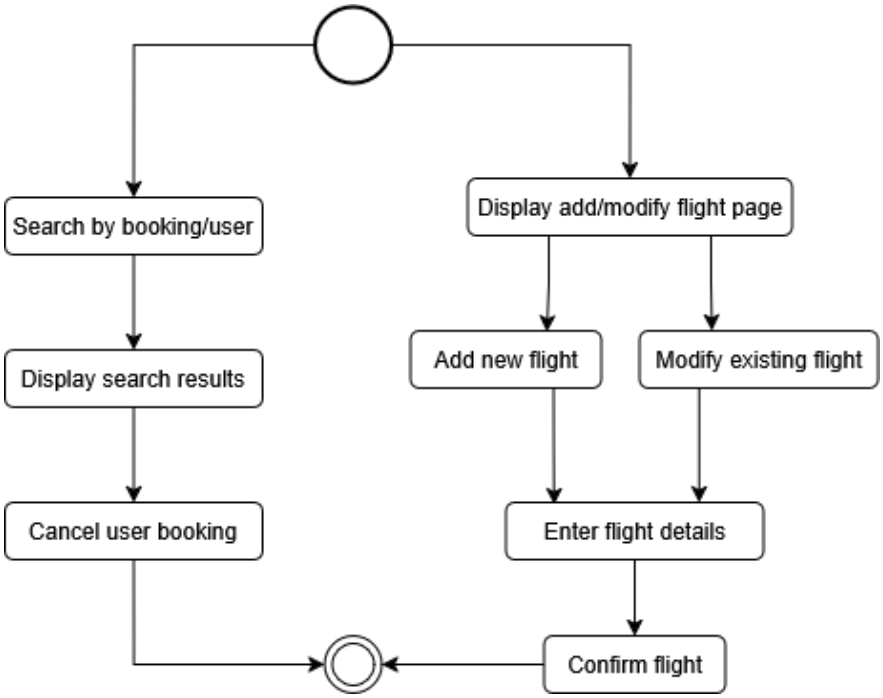

Figure 6: Activity Diagram (Admin Panel)

4.2.6 Class Diagram

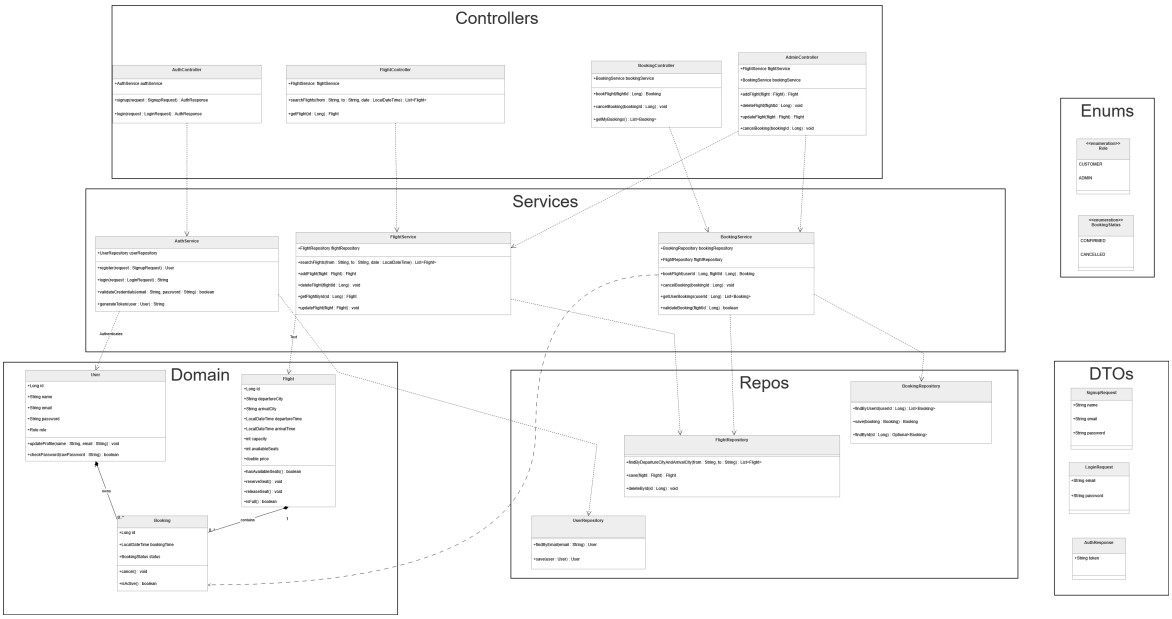

Figure 7: Class Diagram

4.2.7 State Machine Diagrams

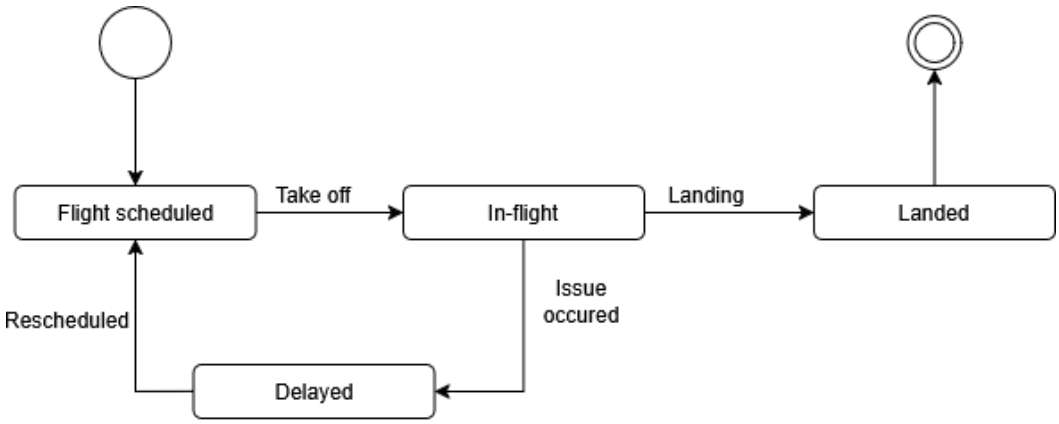

Figure 8: State Machine Diagram (Flight Status)

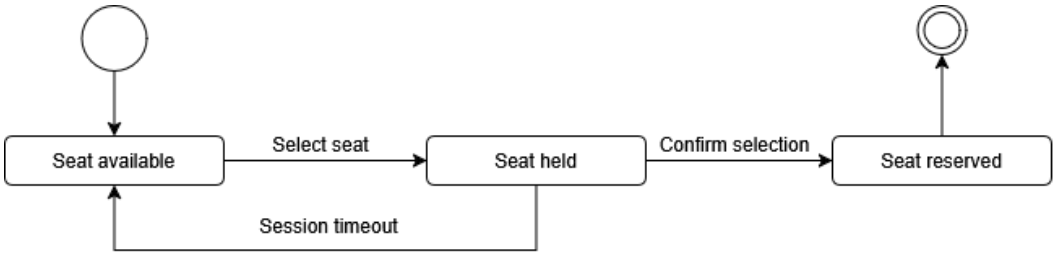

Figure 9: State Machine Diagram (Seat Reservation)

4.3 Implementation Phase

4.3.1 Technologies and Tools Used

Backend

- Java 25
- Spring Boot
- Spring Security
- Spring Data JPA
- Flyway
- PostgreSQL
- JJWT
- Lombok

Frontend

- Next.js
- React + TypeScript
- Tailwind CSS

Tooling and Infrastructure

- Docker + Docker Compose
- Maven Wrapper
- npm

Testing

- JUnit 5
- Mockito
- Spring Boot Test
- Testcontainers
- Bash + curl scripts

4.3.2 Development Overview

The backend was developed as a modular Spring Boot application with clear separation between controllers, services, and repositories.

The frontend was implemented using Next.js with dedicated booking and administration pages.

Database schema creation and seed data are handled using Flyway migrations.

4.3.3 Key Components

- Authentication module.
- User management module.
- Airport management module.
- Airplane and seat module.
- Flight management module.
- Reservation and booking module.
- Ticket management module.
- Background job processing.

4.4 Testing Phase

4.4.1 Testing Strategy

Testing was performed using:

- Unit testing.
- Integration testing.
- API workflow validation scripts.
- Manual UI testing.

4.4.2 Test Cases

ID	Scenario	Steps	Expected Result
TC-1	Customer login	Login with valid credentials	JWT token returned
TC-2	Admin login	Login using admin account	Admin JWT returned
TC-3	Flight search	Search flights	Matching flights returned
TC-4	Seat map retrieval	Request seat map	Seat availability displayed
TC-5	Checkout success	Submit booking	Reservation confirmed
TC-6	Seat occupied	Attempt occupied seat booking	Request rejected
TC-7	Reservation cancellation	Cancel reservation	Reservation cancelled
TC-8	Ticket cancellation	Cancel ticket	Ticket cancelled
TC-9	Invalid payment	Expired card input	Validation error returned
TC-10	Background job	Force completed flight	Status updated correctly

4.4.3 Verification of Correctness

Correctness was verified through:

- Automated testing using Maven.
- Manual booking flow validation.
- API validation scripts.
- Security authorization checks.

4.5 Maintenance Phase

4.5.1 Maintainability

The system is maintainable due to:

- Modular architecture.
- DTO-based APIs.
- Flyway migrations.
- Dockerized deployment.
- Centralized configuration.

4.5.2 Future Improvements

- Real payment gateway integration.
- Email and SMS notifications.
- Advanced flight search filters.
- Analytics dashboards.
- Additional security hardening.

4.5.3 Scalability and Updates

Scalability is supported through:

- Stateless JWT authentication.
- Paginated endpoints.
- Independent PostgreSQL service.
- Database transactions and locking.
- Modular extensible architecture.

5 Project Management and Collaboration

5.1 Trello

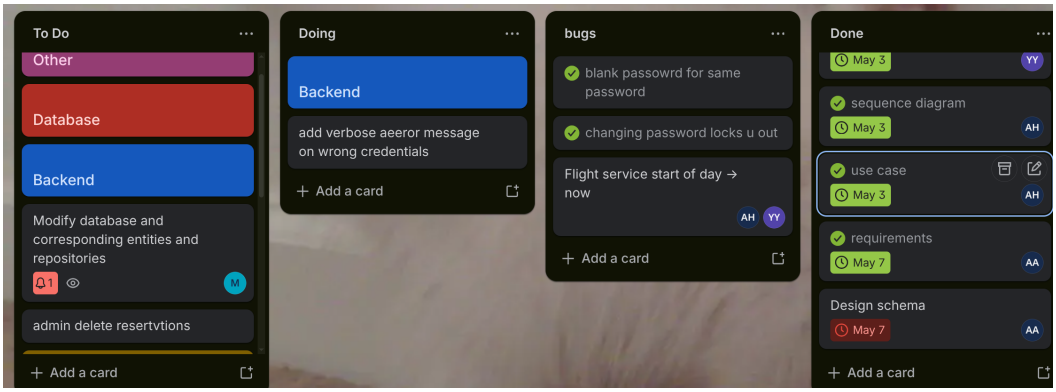

Figure 10: Trello Board Screenshot

Trello was used to track tasks and SDLC progress using workflows such as Backlog, In Progress, Testing, and Done.

5.2 Notion

Notion was used for centralized documentation, meeting notes, requirements, UML references, and testing notes.